Lux Workspace Model: A Z Specification

Formal Model of Frames, Client Identity, and World Menu

March 2026

Contents

1 Introduction

Document status. This is a formal workspace-model specification, not the canonical rewrite- target document. For the rewrite target, start with `target/target.md`. For the current/intermediate whole-system narrative, see `system.tex`.

DES-022 introduces a workspace model for the Lux display server, inspired by Pharo/Squeak's live environment. The display becomes a *workspace* containing independent, movable *frames* (ImGui inner windows). Each frame is owned by a connected client and contains one or more *scenes*.

This specification models the workspace layer that sits above the existing scene renderer (specified in `display-server.tex`). It covers:

- Client identity from the connection handshake
- Frame lifecycle: creation, minimization, restoration, closure
- Frame ownership and cleanup on client disconnect
- World menu: per-client namespaced menu registration
- Scene-to-frame targeting

We abstract away the element tree inside scenes (already modeled) and focus on the new state that DES-022 introduces.

2 Basic Types

$[FD, FRAMEID, SCENEID, CLIENTNAME, MENUID, MENUPATH]$

FD models socket file descriptors (shared with the display-server spec). *FRAMEID* is the identifier a client assigns to a frame. *SCENEID* identifies a scene within a frame. *CLIENTNAME* is the human-readable display name declared during the connection handshake (e.g. "Lux", "Vox", "Quarry"). *MENUID* and *MENUPATH* identify menu items and their nesting path in the World menu.

3 Free Types

$ZBOOL ::= ztrue \mid zfalse$

Frame visibility state:

$FrameVis ::= fvNormal \mid fvMinimized$

Menu target — which menu surface an item belongs to:

$MenuTarget ::= mtTools \mid mtWorld$

4 Global Constants

Animation-friendly bounds. Production values would be larger; these are sized for ProB's finite state space.

$$
\begin{array}{|l}
maxClients : \mathbb{N} \\
maxFrames : \mathbb{N} \\
maxScenes : \mathbb{N} \\
maxMenuItems : \mathbb{N} \\
\hline
maxClients = 3 \\
maxFrames = 4 \\
maxScenes = 6 \\
maxMenuItems = 6
\end{array}
$$

5 Workspace State

The workspace tracks connected clients with their identities, frames with their ownership and visibility, scene-to-frame mappings, and per-client menu registrations. All fields are flattened for ProB animatability.

Workspace

$$
\begin{array}{l}
clients : \mathbb{P}\ FD \\
clientNames : FD \pfun CLIENTNAME \\
frames : \mathbb{P}\ FRAMEID \\
frameOwner : FRAMEID \pfun FD \\
frameVis : FRAMEID \pfun FrameVis \\
frameScenes : FRAMEID \pfun \mathbb{P}\ SCENEID \\
sceneFrame : SCENEID \pfun FRAMEID \\
menuItems : \mathbb{P}\ MENUID \\
menuOwner : MENUID \pfun FD \\
menuTarget : MENUID \pfun MenuTarget \\
menuPath : MENUID \pfun MENUPATH \\
\hline
\dom clientNames \subseteq clients \\
\#clients \leq maxClients \\
\dom frameOwner = frames \\
\dom frameVis = frames \\
\dom frameScenes = frames \\
\ran frameOwner \subseteq clients \\
\#frames \leq maxFrames \\
\dom sceneFrame = \bigcup(\ran frameScenes) \\
\forall f : frames \bullet \forall s : frameScenes\ f \bullet \quad sceneFrame\ s = f \\
\#(\dom sceneFrame) \leq maxScenes \\
\dom menuOwner = menuItems \\
\dom menuTarget = menuItems \\
\dom menuPath = menuItems \\
\ran menuOwner \subseteq clients \\
\#menuItems \leq maxMenuItems
\end{array}
$$

The key invariants are:

1. **Named clients are connected**: Only connected clients can have display names.
2. **Frame owners are connected**: Every frame is owned by a connected client. Disconnecting a client removes its frames.
3. **Frame metadata coverage**: Every frame has an owner, visibility state, and scene set.
4. **Scene-frame consistency**: Every scene knows which frame it belongs to, and that mapping is consistent with the frame's scene set.
5. **Menu owners are connected**: Every menu item is owned by a connected client.

6 Initialization

Init

$$Workspace'$$

$$clients' = \varnothing$$
$$clientNames' = \varnothing$$
$$frames' = \varnothing$$
$$frameOwner' = \varnothing$$
$$frameVis' = \varnothing$$
$$frameScenes' = \varnothing$$
$$sceneFrame' = \varnothing$$
$$menuItems' = \varnothing$$
$$menuOwner' = \varnothing$$
$$menuTarget' = \varnothing$$
$$menuPath' = \varnothing$$

7 Operations

7.1 Accept Connection

A new client connects. It has no display name yet (the `ConnectMessage` arrives later).

AcceptConnection

$$\Delta Workspace$$
$$newClient? : FD$$

$$newClient? \notin clients$$
$$\#clients < maxClients$$
$$clients' = clients \cup \{newClient?\}$$
$$clientNames' = clientNames$$
$$frames' = frames$$
$$frameOwner' = frameOwner$$
$$frameVis' = frameVis$$
$$frameScenes' = frameScenes$$
$$sceneFrame' = sceneFrame$$
$$menuItems' = menuItems$$
$$menuOwner' = menuOwner$$
$$menuTarget' = menuTarget$$
$$menuPath' = menuPath$$

7.2 Identify Client

A connected client declares its display name via `ConnectMessage`. Subsequent calls update the name (idempotent).

IdentifyClient

$\Delta Workspace$
$fd? : FD$
$name? : CLIENTNAME$

$fd? \in clients$
$clientNames' = clientNames \oplus \{fd? \mapsto name?\}$
$clients' = clients$
$frames' = frames$
$frameOwner' = frameOwner$
$frameVis' = frameVis$
$frameScenes' = frameScenes$
$sceneFrame' = sceneFrame$
$menuItems' = menuItems$
$menuOwner' = menuOwner$
$menuTarget' = menuTarget$
$menuPath' = menuPath$

7.3 Create Frame

A client sends a `SceneMessage` with a `frame_id` that does not yet exist. The display creates a new frame containing the specified scene.

CreateFrame

$\Delta Workspace$
$fd? : FD$
$frameId? : FRAMEID$
$sceneId? : SCENEID$

$fd? \in clients$
$frameId? \notin frames$
$sceneId? \notin \mathrm{dom}\ sceneFrame$
$\#frames < maxFrames$
$\#(\mathrm{dom}\ sceneFrame) < maxScenes$
$frames' = frames \cup \{frameId?\}$
$frameOwner' = frameOwner \cup \{frameId? \mapsto fd?\}$
$frameVis' = frameVis \cup \{frameId? \mapsto fvNormal\}$
$frameScenes' = frameScenes \cup \quad \{frameId? \mapsto \{sceneId?\}\}$
$sceneFrame' = sceneFrame \cup \{sceneId? \mapsto frameId?\}$
$clients' = clients$
$clientNames' = clientNames$
$menuItems' = menuItems$
$menuOwner' = menuOwner$
$menuTarget' = menuTarget$
$menuPath' = menuPath$

7.4 Add Scene to Frame

A client sends a `SceneMessage` targeting an existing frame with a new scene. The scene is added (creating a tab if this is the second scene in the frame).

AddSceneToFrame

$\Delta Workspace$
$fd? : FD$
$frameId? : FRAMEID$
$sceneId? : SCENEID$

$fd? \in clients$
$frameId? \in frames$
$frameOwner\, frameId? = fd?$
$sceneId? \notin \mathrm{dom}\, sceneFrame$
$\#(\mathrm{dom}\, sceneFrame) < maxScenes$
$frameScenes' = frameScenes \oplus \{frameId? \mapsto (frameScenes\, frameId? \cup \{sceneId?\})\}$
$sceneFrame' = sceneFrame \cup \{sceneId? \mapsto frameId?\}$
$frames' = frames$
$frameOwner' = frameOwner$
$frameVis' = frameVis$
$clients' = clients$
$clientNames' = clientNames$
$menuItems' = menuItems$
$menuOwner' = menuOwner$
$menuTarget' = menuTarget$
$menuPath' = menuPath$

7.5 Update Scene in Frame

A client sends a `SceneMessage` targeting an existing frame with an existing scene ID. The scene content is replaced (modeled here as a no-op on workspace state since scene *content* is in the display-server spec).

UpdateScene

$\Xi Workspace$
$fd? : FD$
$frameId? : FRAMEID$
$sceneId? : SCENEID$

$fd? \in clients$
$frameId? \in frames$
$frameOwner\, frameId? = fd?$
$sceneId? \in frameScenes\, frameId?$

7.6 Close Frame (User)

The user clicks the close button on a frame. The frame and all its scenes are removed. An interaction event is sent to the owning client (modeled at the display-server level).

CloseFrame

$\Delta Workspace$
$frameId? : FRAMEID$

$frameId? \in frames$
$frames' = frames \setminus \{frameId?\}$
$frameOwner' = \{frameId?\} \lhd frameOwner$
$frameVis' = \{frameId?\} \lhd frameVis$
$frameScenes' = \{frameId?\} \lhd frameScenes$
$sceneFrame' = (frameScenes\, frameId?) \lhd sceneFrame$
$clients' = clients$
$clientNames' = clientNames$
$menuItems' = menuItems$
$menuOwner' = menuOwner$
$menuTarget' = menuTarget$
$menuPath' = menuPath$

7.7 Minimize Frame

MinimizeFrame

$\Delta Workspace$
$frameId? : FRAMEID$

$frameId? \in frames$
$frameVis\, frameId? = fvNormal$
$frameVis' = frameVis \oplus \{frameId? \mapsto fvMinimized\}$
$frames' = frames$
$frameOwner' = frameOwner$
$frameScenes' = frameScenes$
$sceneFrame' = sceneFrame$
$clients' = clients$
$clientNames' = clientNames$
$menuItems' = menuItems$
$menuOwner' = menuOwner$
$menuTarget' = menuTarget$
$menuPath' = menuPath$

7.8 Restore Frame

RestoreFrame

$\Delta Workspace$
$frameId? : FRAMEID$

$frameId? \in frames$
$frameVis\, frameId? = fvMinimized$
$frameVis' = frameVis \oplus \{frameId? \mapsto fvNormal\}$
$frames' = frames$
$frameOwner' = frameOwner$
$frameScenes' = frameScenes$
$sceneFrame' = sceneFrame$
$clients' = clients$
$clientNames' = clientNames$
$menuItems' = menuItems$
$menuOwner' = menuOwner$
$menuTarget' = menuTarget$
$menuPath' = menuPath$

7.9 Register Menu Item

A client registers a menu item. The `menuTarget` determines whether it appears in the Tools menu or the World menu (per-client namespace from handshake identity).

RegisterMenuItem

$$
\begin{array}{l}
\Delta Workspace \\
fd? : FD \\
itemId? : MENUID \\
target? : MenuTarget \\
path? : MENUPATH \\
\hline
fd? \in clients \\
itemId? \notin menuItems \\
\#menuItems < maxMenuItems \\
menuItems' = menuItems \cup \{itemId?\} \\
menuOwner' = menuOwner \cup \{itemId? \mapsto fd?\} \\
menuTarget' = menuTarget \cup \{itemId? \mapsto target?\} \\
menuPath' = menuPath \cup \{itemId? \mapsto path?\} \\
clients' = clients \\
clientNames' = clientNames \\
frames' = frames \\
frameOwner' = frameOwner \\
frameVis' = frameVis \\
frameScenes' = frameScenes \\
sceneFrame' = sceneFrame
\end{array}
$$

7.10 Deregister Menu Item

A client removes a menu item it owns.

DeregisterMenuItem

$$
\begin{array}{l}
\Delta Workspace \\
fd? : FD \\
itemId? : MENUID \\
\hline
fd? \in clients \\
itemId? \in menuItems \\
menuOwner\ itemId? = fd? \\
menuItems' = menuItems \setminus \{itemId?\} \\
menuOwner' = \{itemId?\} \lhd menuOwner \\
menuTarget' = \{itemId?\} \lhd menuTarget \\
menuPath' = \{itemId?\} \lhd menuPath \\
clients' = clients \\
clientNames' = clientNames \\
frames' = frames \\
frameOwner' = frameOwner \\
frameVis' = frameVis \\
frameScenes' = frameScenes \\
sceneFrame' = sceneFrame
\end{array}
$$

7.11 Disconnect Client

A client disconnects. All its frames, scenes, and menu items are removed. This is the critical cleanup operation that maintains the ownership invariants.

Let *deadFrames* be the set of frames owned by the departing client, and *deadScenes* be all scenes in those frames, and *deadMenus* be all menu items owned by the client.

DisconnectClient

$\Delta Workspace$
$deadClient? : FD$

$deadClient? \in clients$
$clients' = clients \setminus \{deadClient?\}$
$clientNames' = \{deadClient?\} \lhd clientNames$
$frames' = frames \setminus \{f : frames \mid frameOwner\, f = deadClient?\}$
$frameOwner' = frames' \lhd frameOwner$
$frameVis' = frames' \lhd frameVis$
$frameScenes' = frames' \lhd frameScenes$
$sceneFrame' = sceneFrame \rhd frames'$
$menuItems' = menuItems \setminus \{m : menuItems \mid menuOwner\, m = deadClient?\}$
$menuOwner' = menuItems' \lhd menuOwner$
$menuTarget' = menuItems' \lhd menuTarget$
$menuPath' = menuItems' \lhd menuPath$

8 System Invariants Summary

The key properties maintained across all operations:

1. **Frame owners are connected**: $\operatorname{ran} frameOwner \subseteq clients$. No orphaned frames.
2. **Scene-frame consistency**: Every scene maps to exactly one frame, and that frame lists the scene in its scene set. No orphaned scenes.
3. **Menu owners are connected**: $\operatorname{ran} menuOwner \subseteq clients$. No orphaned menu items.
4. **Disconnect atomicity**: When a client disconnects, all its frames, scenes, and menu items are removed in one atomic operation.
5. **Bounded resources**: Client count, frame count, scene count, and menu item count are all bounded.
6. **Named clients are connected**: $\operatorname{dom} clientNames \subseteq clients$. Names cannot outlive connections.
7. **Ownership exclusivity**: Each frame has exactly one owner (*frameOwner* is a total function on *frames*). Only the owning client can add scenes to a frame.